Heap Data Structure

Introduction | Insertion | Deletion — Step by Step

Max-Heap & Min-Heap

PART 1 — Introduction to Heap

1. What is a Heap?

A Heap is a special Complete Binary Tree that satisfies two properties simultaneously:

Property	Description
Shape Property	A heap is a Complete Binary Tree — all levels fully filled except possibly the last, which is filled LEFT to RIGHT.
Heap Property	Max-Heap: every parent >= its children. Min-Heap: every parent <= its children.

> **Key Point:** A heap is NOT a sorted structure. It only guarantees the ROOT holds the maximum (Max-Heap) or minimum (Min-Heap). Sibling ordering is not defined.

1.1 Types of Heap

Type	Root contains	Parent vs Children	Common Uses
Max-Heap	Largest element	Parent >= both children	Priority queues, HeapSort
Min-Heap	Smallest element	Parent <= both children	Dijkstra, scheduling

1.2 Max-Heap Example

A valid Max-Heap with 9 elements:

```
                    (100)                    <- Root (largest)
                   /     \
              (80)       (90)
             /   \      /   \
         (50)   (70)  (60)   (85)
         /  \
     (30)   (40)
```

Figure 1 — Max-Heap tree (every parent >= its children)

Array representation (0-indexed):

Array:	100	80	90	50	70	60	85	30	40
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

Index 0 = root. Index 0 always holds the max value in a Max-Heap.

1.3 Min-Heap Example

A valid Min-Heap with 9 elements:

```
                    (10)                     <- Root (smallest)
                   /     \
              (20)       (15)
             /   \      /   \
         (50)   (30)  (40)   (25)
         /  \
     (80)   (60)
```

Figure 2 — Min-Heap tree (every parent <= its children)

1.4 Heap as a 0-Indexed Array

A heap is stored as a 0-indexed array. For any node at index i:

Parent of i: `floor((i - 1) / 2)` (valid when i > 0)
Left child of i: `2 * i + 1`
Right child of i: `2 * i + 2`

> **Last non-leaf node:** `floor((n - 2) / 2)  =  floor(n/2) - 1`
> **Leaf nodes:** `floor(n/2)  to  n-1`
> **Root:** `index 0`

Example — mapping the Max-Heap tree to a 0-indexed array:

Index	0	1	2	3	4	5	6	7	8
Value	100	80	90	50	70	60	85	30	40
Node	Root	L of [0]	R of [0]	L of [1]	R of [1]	L of [2]	R of [2]	L of [3]	R of [3]

1.5 Index Formula Verification

Let us verify the 0-indexed formulas on the Max-Heap:

Node (index)	Value	Left child = 2i+1	Right child = 2i+2	Parent = floor((i-1)/2)
i = 0 (root)	100	1 (80)	2 (90)	— (no parent)
i = 1	80	3 (50)	4 (70)	0 (100)
i = 2	90	5 (60)	6 (85)	0 (100)
i = 3	50	7 (30)	8 (40)	1 (80)
i = 7	30	15 (none)	16 (none)	3 (50)

1.6 Properties Summary

Property	Value
Structure	Complete Binary Tree
Storage	0-indexed array — no pointers, cache friendly
Root	Index 0 — holds max (Max-Heap) or min (Min-Heap)
Height	floor($\log_2 n$) → O(log n)

Parent of node i	floor((i − 1) / 2)
Left child of i	2i + 1
Right child of i	2i + 2
Last non-leaf	floor(n/2) − 1
Leaf indices	floor(n/2) to n−1
Space	O(n)

PART 2 — Insertion in a Heap (HEAPIFY-UP)

2. Heap Insertion

Key Idea: Add the new element at the END of the array (index n) to maintain the Complete Binary Tree shape. Then HEAPIFY-UP (Bubble Up) — compare with parent and swap upward until heap property holds.

Operation: HEAPIFY-UP — while i > 0 and heap[i] violates order with heap[parent]: SWAP and move up.

2.1 Insertion Algorithm (0-indexed)

```
INSERT(heap, key):
        heap[n] = key                     // place at index n (end)
        n = n + 1                         // increase heap size
        HEAPIFY_UP(heap, n - 1)           // start at new last index

HEAPIFY_UP(heap, i):
        while i > 0:
          parent = floor((i - 1) / 2)
          if heap[i] > heap[parent]:      // For Max-Heap
              SWAP(heap[i], heap[parent])
              i = parent                  // move up
          else:
              break                       // heap property restored
```

Time Complexity: O(log n) — new element bubbles up at most h = floor($\log_2$ n) levels.

2.2 Max-Heap Insertion — Step by Step

Starting Max-Heap with n=6 elements:

```
                (90)
               /    \
            (80)    (70)
           /   \    /
        (50) (60) (40)
```

Figure 3 — Max-Heap before insertion

Array:

90	80	70	50	60	40
[0]	[1]	[2]	[3]	[4]	[5]

Insert key = 85. n becomes 7.

Step 1: Place 85 at index 6 (end of array)

heap[6] = 85. n = 7. Complete Binary Tree shape is maintained.

```
                (90)
               /    \
            (80)    (70)
           /   \    /  \
        (50) (60) (40) >>>(85)<<<   <- inserted at index 6
```

Array:

90	80	70	50	60	40	85
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Call HEAPIFY_UP at i = 6.

Step 2: HEAPIFY-UP: i=6, compare with parent

parent = floor((6-1)/2) = floor(5/2) = 2 → heap[2] = 70

heap[6]=85 > heap[2]=70 → SWAP

```
                (90)
               /    \
            (80)   >>>(85)<<<     <- swapped up
           /   \    /  \
        (50) (60) (40) (70)       <- moved down
```

Array:

90	80	**85**	50	60	40	70
[0]	[1]	[2]	[3]	[4]	[5]	[6]

i = parent = 2. Continue HEAPIFY-UP.

Step 3: HEAPIFY-UP: i=2, compare with parent

parent = floor((2-1)/2) = floor(1/2) = 0 → heap[0] = 90

heap[2]=85 < heap[0]=90 → NO SWAP. Stop.

```
                (90)
               /    \
            (80)    (85)          <- final position
           /   \    /  \
        (50) (60) (40) (70)
```

Figure 4 — Max-Heap after inserting 85

Array:

90	85	80	50	60	40	70
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Insertion complete! 85 is at index 2.

2.3 Insertion Trace Table

Iteration	i	parent=floor((i-1)/2)	heap[i]	heap[parent]	Condition	Action
1	6	floor(5/2) = 2	85	70	85 > 70	SWAP → i=2
2	2	floor(1/2) = 0	85	90	85 < 90	STOP

2.4 Min-Heap Insertion — Step by Step

Starting Min-Heap with n=6 elements:

```
                (10)
               /    \
            (20)    (15)
           /    \   /
        (50)  (30)(40)
```

Figure 5 — Min-Heap before insertion

Array:

10	20	15	50	30	40
[0]	[1]	[2]	[3]	[4]	[5]

Insert key = 5. n becomes 7.

Step 1: Place 5 at index 6

heap[6] = 5. n = 7.

Array:

10	20	15	50	30	40	5
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Call HEAPIFY_UP at i = 6.

Step 2: HEAPIFY-UP: i=6, parent=floor(5/2)=2, heap[2]=15

5 < 15 → SWAP (Min-Heap: swap if child < parent).

Array:

10	20	5	50	30	40	15
[0]	[1]	[2]	[3]	[4]	[5]	[6]

i = 2. Continue.

Step 3: **HEAPIFY-UP: i=2, parent=floor(1/2)=0, heap[0]=10**

5 < 10 → SWAP.

Array:

5	20	10	50	30	40	15
[0]	[1]	[2]	[3]	[4]	[5]	[6]

i = 0 = root. **Stop. Insertion complete.**

```
                 (5)              <- new root (smallest)
              /     \
           (20)     (10)
          /    \    /  \
       (50)  (30) (40) (15)
```

Figure 6 — Min-Heap after inserting 5

PART 3 — Deletion in a Heap (HEAPIFY-DOWN)

3. Heap Deletion

Key Idea: Only the ROOT (index 0) is deleted — it holds the max or min. Steps:

- **Save heap[0]** (the element to return).
- **Replace heap[0]** with heap[n-1] (the last element).
- **Reduce n by 1.**
- **HEAPIFY-DOWN** — push the new root down to its correct position.

3.1 Deletion Algorithm (0-indexed)

```
DELETE_MAX(heap):   // Max-Heap
        max = heap[0]               // save root
        heap[0] = heap[n-1]         // replace root with last element
        n = n - 1                   // shrink heap
        HEAPIFY_DOWN(heap, 0)       // restore heap property
        return max

HEAPIFY_DOWN(heap, i):
        while True:
          left  = 2 * i + 1
          right = 2 * i + 2
          largest = i

          if left < n and heap[left] > heap[largest]:
              largest = left
          if right < n and heap[right] > heap[largest]:
              largest = right

          if largest != i:
              SWAP(heap[i], heap[largest])
              i = largest
          else:
              break
```

Time Complexity: O(log n) — element sifts down at most h = floor($\log_2$ n) levels.

3.2 Max-Heap Deletion — Step by Step

Delete the maximum from this Max-Heap (n=9):

```
                    (100)
                   /     \
              (80)        (90)
             /   \       /   \
         (50)   (70)  (60)   (85)
         /   \
     (30)   (40)
```

Figure 7 — Max-Heap before deletion

Array:

100	80	90	50	70	60	85	30	40
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

Step 1: **Remove root — replace heap[0] with last element heap[8]=40; n=8**

Saved max = 100. heap[0] = heap[8] = **40**. n = 8.

```
                    >>>(40)<<<   <- placed at root (index 0)
                   /     \
              (80)        (90)
             /   \       /   \
         (50)   (70)  (60)   (85)
         /
     (30)                [40 removed from index 8]
```

Array:

40	80	90	50	70	60	85	30
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

Call HEAPIFY_DOWN at i = 0.

Step 2: **HEAPIFY-DOWN: i=0 (value 40) — find largest child**

left = 2*0+1 = 1 → heap[1] = 80

right = 2*0+2 = 2 → heap[2] = 90

Largest = index 2 (value 90). 40 < 90 → SWAP heap[0] and heap[2].

```
                    (90)          <- swapped up to index 0
                   /    \
              (80)       >>>(40)<<<  <- moved down to index 2
             /   \      /   \
         (50)  (70) (60)  (85)
         /
     (30)
```

Array:

90	80	**40**	50	70	60	85	30
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

i = 2. Continue.

Step 3: HEAPIFY-DOWN: i=2 (value 40) — find largest child

left = 2*2+1 = 5 → heap[5] = 60

right = 2*2+2 = 6 → heap[6] = 85

Largest = index 6 (value 85). 40 < 85 → SWAP heap[2] and heap[6].

```
                    (90)
                   /    \
              (80)       (85)    <- swapped up to index 2
             /   \      /   \
         (50)  (70)(60)  >>>(40)<<<  <- at index 6
         /
     (30)
```

Array:

90	80	85	50	70	60	**40**	30
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

i = 6. Continue.

Step 4: HEAPIFY-DOWN: i=6 (value 40) — check for children

left = 2*6+1 = 13 → 13 >= n=8. No left child.

right = 2*6+2 = 14 → 14 >= n=8. No right child.

No children. STOP. 40 is at a leaf. Heap fully restored.

```
                    (90)          <- new root (max)
                   /    \
              (80)       (85)
             /   \      /   \
         (50)   (70) (60)   (40)
         /
     (30)
```

Figure 8 — Max-Heap after deleting 100

Array:

90	80	85	50	70	60	40	30
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

Deleted: 100. New max = 90 at index 0.

3.3 Deletion Trace Table

Iter	i	left=2i+1	right=2i+2	Largest	heap[i]	Action
1	0 (40)	1 (80)	2 (90)	2 (90)	40	SWAP → i=2
2	2 (40)	5 (60)	6 (85)	6 (85)	40	SWAP → i=6
3	6 (40)	13 (none)	14 (none)	6 (40)	40	STOP (leaf)

3.4 Min-Heap Deletion — Step by Step

Delete the minimum from this Min-Heap (n=7):

```
                (5)
              /     \
          (20)      (10)
         /    \    /   \
      (50)  (30) (40) (15)
```

Figure 9 — Min-Heap before deletion

Array:

5	20	10	50	30	40	15
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Step 1: Remove root — replace heap[0] with last element heap[6]=15; n=6

Saved min = 5. heap[0] = heap[6] = **15**. n = 6.

```
              >>>(15)<<<   <- placed at root
              /     \
          (20)      (10)
         /    \    /
      (50)  (30) (40)
```

Array:

15	20	10	50	30	40
[0]	[1]	[2]	[3]	[4]	[5]

Call HEAPIFY_DOWN at i = 0.

Step 2: HEAPIFY-DOWN: i=0 (15) — find smallest child

left = 1 → heap[1] = 20

right = 2 → heap[2] = 10

Smallest = index 2 (value 10). 15 > 10 → SWAP

```
              (10)          <- swapped up to index 0
              /     \
          (20)      >>>(15)<<<  <- moved to index 2
         /    \    /
      (50)  (30) (40)
```

Array:

10	20	15	50	30	40
[0]	[1]	[2]	[3]	[4]	[5]

i = 2. Continue.

Step 3: HEAPIFY-DOWN: i=2 (15) — find smallest child

left = 2*2+1 = 5 → heap[5] = 40

right = 2*2+2 = 6 → 6 >= n=6. No right child.

Smallest = index 5 (value 40). 15 < 40 → NO SWAP. STOP.

```
                (10)          <- new root (min)
               /    \
            (20)    (15)
           /    \   /
        (50) (30)(40)
```

Figure 10 — Min-Heap after deleting 5

Array:

10	20	15	50	30	40
[0]	[1]	[2]	[3]	[4]	[5]

Deleted: 5. New min = 10.

4. Max-Heap vs Min-Heap — Full Comparison

Aspect	Max-Heap	Min-Heap
Root (index 0)	Maximum element	Minimum element
Heap property	heap[parent] >= heap[child]	heap[parent] <= heap[child]
HEAPIFY-UP swap condition	heap[i] > heap[parent]	heap[i] < heap[parent]
HEAPIFY-DOWN target	Largest child	Smallest child
Extract operation	Extract max in O(1)	Extract min in O(1)
Common use	Priority queue (max priority first)	Priority queue (min priority first)

5. Time & Space Complexity

Operation	Time Complexity	Space	Notes
Build Heap	O(n)	O(n)	Run HEAPIFY-DOWN from index floor(n/2)-1 down to 0
INSERT	O(log n)	O(1)	Bubble up at most floor($\log_2$ n) levels
DELETE (root)	O(log n)	O(1)	Sift down at most floor($\log_2$ n) levels
PEEK (get max/min)	O(1)	O(1)	Just read heap[0]
SEARCH arbitrary key	O(n)	O(1)	No ordering except root
HEAP SORT	O(n log n)	O(1)	Build + n deletions

6. Key Properties — Quick Reference

Property	Detail
Array start index	0 (root is at index 0)
Parent of node i	floor((i − 1) / 2)
Left child of i	2i + 1
Right child of i	2i + 2
Root (max/min)	Always at index 0
Last non-leaf	floor(n/2) − 1
Leaf nodes	Indices floor(n/2) to n−1
INSERT	Append at index n → HEAPIFY-UP
DELETE	Replace index 0 with heap[n-1], shrink n → HEAPIFY-DOWN
HEAPIFY-UP	Swap with parent while heap property violated
HEAPIFY-DOWN	Swap with largest (Max) or smallest (Min) child while violated
Height	Always O(log n) — Complete Binary Tree guarantee